IMPLEMENTASI WORDCOUNT DENGAN PYSPARK DAN ANALISIS PERFORMA PEMROSESAN PARALEL

Asep Suhaedi (NIM 312410230)
Teknik Informatika, Universitas Pelita Bangsa
Mata Kuliah Big Data | 06-10-2026

1. PENDAHULUAN

WordCount adalah kasus dasar pemrosesan data paralel, yaitu menghitung frekuensi setiap kata pada kumpulan teks. Apache Spark memproses data sebagai *Resilient Distributed Dataset* (RDD) yang dipartisi ke banyak executor sehingga komputasi berjalan paralel dan tahan gagal melalui lineage [1]. Tugas ini mengimplementasikan WordCount dengan PySpark, memetakan alur Job, Stage, dan Task, membandingkan pemrosesan single machine dengan Spark, mengidentifikasi bottleneck, serta menganalisis pengaruh jumlah partition terhadap performa.

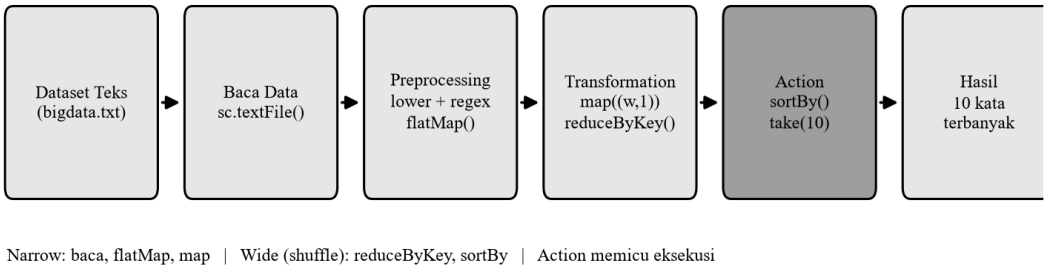
2. LINGKUNGAN DAN DATASET

Tabel 1. Lingkungan pengujian

Komponen	Nilai
OS / CPU / RAM	Windows 11 / 8 core logis / 7.3 GB
Python / PySpark	3.12.10 / 4.2.0
Mode Spark	local[*] (semua core satu mesin; bukan cluster multi-node)
Dataset	100,000 baris, 10.4 MB, teks sintetis berdistribusi Zipf (seed tetap)
Metode ukur	Median hasil jalan setelah warm-up; waktu dari pembacaan data sampai top-10 tersedia

Dataset sintetis dipilih agar eksperimen dapat direproduksi dan ukurannya dapat diatur. Frekuensi kata mengikuti hukum Zipf sehingga kata fungsi (yang, dan, di) mendominasi, serupa teks bahasa Indonesia nyata.

3. IMPLEMENTASI DAN PIPELINE



Gambar 1. Diagram pipeline pemrosesan data

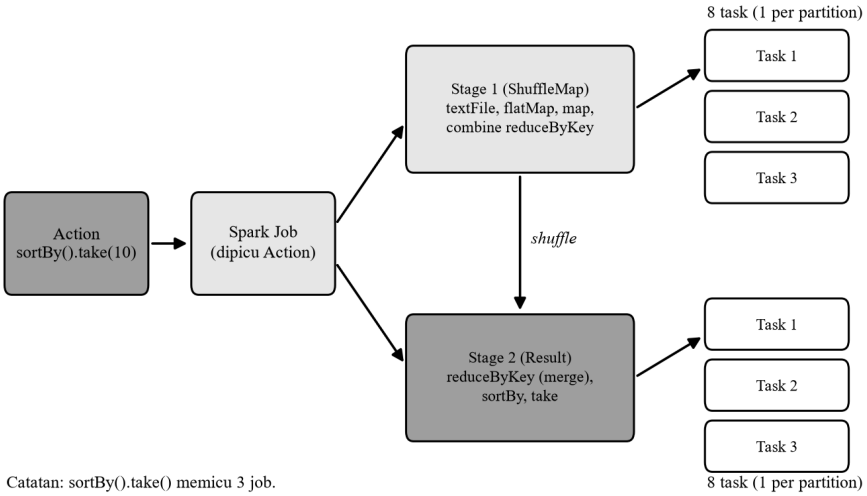
```
lines  = sc.textFile(path, minPartitions=n)
words  = lines.flatMap(lambda l: re.findall(r"[a-z0-9]+", l.lower()))
pairs  = words.map(lambda w: (w, 1))
counts = pairs.reduceByKey(lambda a, b: a + b, numPartitions=n)
top10  = counts.sortBy(lambda kv: (-kv[1], kv[0])).take(10)   # ACTION
```

Tabel 2. Transformation dan action yang digunakan

Operasi	Jenis	Penjelasan
textFile()	Transformation (lazy)	Membaca file teks menjadi RDD baris dan menentukan jumlah partition awal.
flatMap(tokenize)	Transformation, narrow	Mengubah tiap baris menjadi banyak kata (huruf kecil, tanda baca dibuang).
map(w -> (w,1))	Transformation, narrow	Membentuk pasangan key-value untuk penjumlahan.
reduceByKey(a+b)	Transformation, wide	Menjumlahkan nilai per kata; combine di sisi map sebelum shuffle.
sortBy(-count)	Transformation, wide	Mengurutkan global menurun; memicu sampling dan shuffle.
take(10)	Action	Mengambil 10 elemen ke driver; memicu eksekusi seluruh lineage.
count()	Action	Menghitung jumlah baris (validasi dataset).

4. ALUR SPARK JOB, STAGE, DAN TASK

Transformation bersifat lazy; Spark hanya membangun DAG sampai Action dipanggil. Batas Stage ditentukan oleh operasi wide (shuffle): operasi narrow (textFile, flatMap, map, dan combine reduceByKey) berjalan berantai dalam satu Stage, sedangkan reduceByKey sisi reduce dan sortBy membutuhkan data dari semua partition sehingga membentuk Stage baru. Setiap Stage dibagi menjadi Task, satu Task per partition [1][2].



Gambar 2. Alur Action, Job, Stage, dan Task

Tabel 3. Job, Stage, dan Task hasil pengamatan (statusTracker)

Job	Stage	Task	Selesai
0	0: reduceByKey	8	8
0	1: sortBy	8	8
1	2: reduceByKey (dilewati)	8	0
1	3: sortBy	8	8
2	4: reduceByKey (dilewati)	8	0
2	5: sortBy	8	8
2	6: runJob	1	1

Satu pemanggilan `sortBy().take()` menghasilkan 3 Job: sortBy menjalankan job sampling untuk menentukan batas range partition sebelum job sort dan take. Stage yang dilewati adalah Stage reduceByKey yang keluaran shuffle-nya sudah tersedia dari job sebelumnya (reuse).

5. HASIL

5.1 Sepuluh kata terbanyak

Tabel 4. Sepuluh kata dengan frekuensi tertinggi

Peringkat	Kata	Frekuensi
1	yang	223,118
2	dan	107,021
3	di	69,614
4	ke	51,967
5	dari	41,097
6	untuk	34,134
7	dengan	28,839
8	pada	25,117
9	ini	22,147
10	itu	19,724

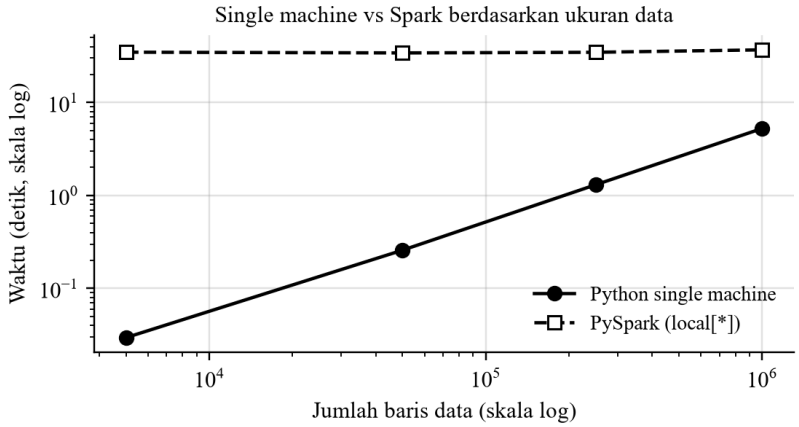
Hasil Spark identik dengan hasil Python murni (diverifikasi otomatis: **cocok** pada semua konfigurasi partition).

5.2 Single machine vs distributed

Tabel 5. Perbandingan single machine dan distributed

Aspek	Single machine (Python)	Distributed (Spark)
Data yang diproses	Terbatas RAM satu mesin	Dibagi ke banyak partition/node, dapat melebihi RAM satu mesin
Waktu (data ini)	0.52 detik	3.89 detik (partition = 1, local[*])
Skalabilitas	Vertikal (upgrade mesin)	Horizontal (tambah node/core)
Ketahanan gagal	Rendah (ulang dari awal)	Tinggi (task diulang dari lineage RDD)
Overhead	Sangat kecil	Start JVM, serialisasi, scheduling, shuffle
Biaya	Murah, sederhana	Lebih mahal (cluster), operasional kompleks

Pada dataset 100,000 baris, Python murni menyelesaikan WordCount dalam **0.52 detik**, sedangkan Spark terbaik (partition = 1) membutuhkan **3.89 detik** (sekitar 7.5 kali lebih lambat). Seluruh data muat di RAM satu mesin sehingga overhead Spark (start JVM, serialisasi Python-JVM, penjadwalan task, shuffle) lebih besar daripada keuntungan paralelisme.

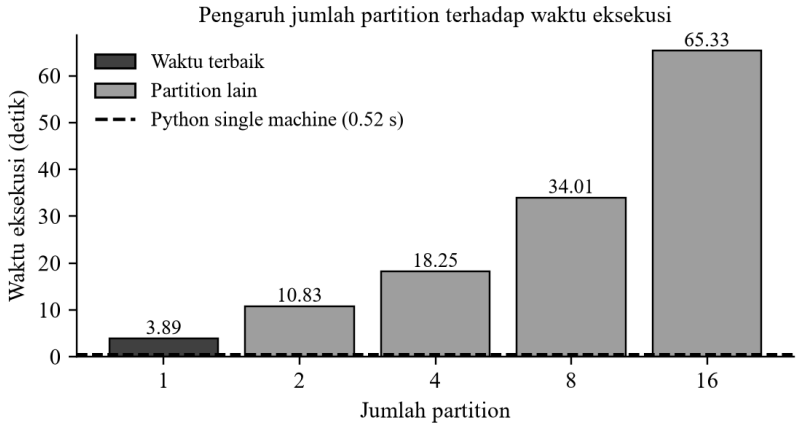


Gambar 3. Waktu eksekusi terhadap ukuran data

Pada seluruh ukuran yang diuji (hingga 1,000,000 baris, 105 MB) Python murni masih lebih cepat. Keunggulan Spark baru muncul ketika data melebihi kapasitas memori satu mesin atau ketika tersedia banyak node/core; pengujian ini hanya memakai satu mesin.

6. ANALISIS PERFORMA

6.1 Pengaruh jumlah partition



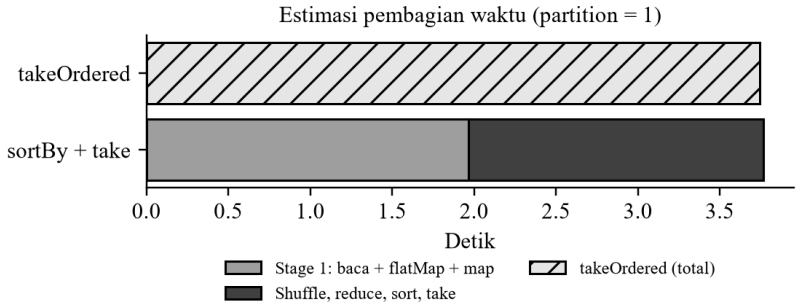
Gambar 4. Waktu eksekusi untuk berbagai jumlah partition

Partition	1	2	4	8	16
Waktu (detik)	3.89	10.83	18.25	34.01	65.33

Waktu tercepat dicapai pada **1 partition** (3.89 detik). Mesin ini memiliki 8 core logis, sehingga menambah partition menambah jumlah task, biaya penjadwalan, dan blok shuffle tanpa menambah paralelisme yang sebanding; waktu terburuk terjadi pada 16 partition (65.33 detik). Ini menunjukkan overhead akibat partition berlebih.

Spark dengan **local[1]** (1 thread, 1 partition) membutuhkan 3.82 detik, dibanding 3.89 detik pada local[*] dengan partition terbaik: tidak ada percepatan berarti (selisih dalam rentang variasi pengukuran).

6.2 Identifikasi bottleneck



Gambar 5. Estimasi pembagian waktu per tahap (selisih total dan Stage 1, bersifat pendekatan)

Dengan 1 partition, tahap baca + flatMap + map (Stage 1, murni narrow) memakan sekitar **52%** dari total waktu, sisanya (48%) berasal dari shuffle, reduceByKey sisi reduce, sortBy, dan take. Mengganti `sortBy().take(10)` dengan `takeOrdered(10)` menghasilkan total

3.74 detik (lebih cepat, rasio 1.01) karena tidak perlu job sampling RangePartitioner dan sort global seluruh kata.

- **Shuffle:** reduceByKey dan sortBy memindahkan data antar partition; biaya naik bersama jumlah partition.
- **Overhead Python-JVM:** lambda Python dijalankan di proses worker Python sehingga tiap record diserialisasi bolak-balik.
- **Data skew:** kata dominan seperti 'yang' sangat sering muncul; map-side combine pada reduceByKey meredamnya.
- **Jumlah partition:** terlalu sedikit membuat core menganggur, terlalu banyak membuat overhead task melebihi kerja nyata.
- **Sorting global tak perlu:** hanya 10 kata dibutuhkan, sehingga takeOrdered cukup dibanding sortBy seluruh kata.

7. KESIMPULAN

- WordCount berhasil diimplementasikan dengan PySpark dan 10 kata terbanyak identik dengan hasil Python murni.
- Pada 100,000 baris, Spark terbaik 3.89 detik (partition = 1) dibanding Python murni 0.52 detik. Spark belum unggul karena overhead lebih besar daripada beban kerja.
- Jumlah partition berpengaruh jelas: 3.89 detik pada 1 partition, 3.89 detik pada titik terbaik, dan 65.33 detik pada 16 partition.
- Bottleneck utama adalah shuffle (reduceByKey, sortBy), overhead serialisasi Python-JVM, dan skew kata dominan.
- Keterbatasan: pengujian memakai mode local[*] pada satu mesin dan data sintetis; hasil pada cluster multi-node dan teks nyata dapat berbeda.

REFERENSI

[1] Zaharia, M., et al. (2012). Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing. NSDI '12.

[2] Apache Spark Documentation. RDD Programming Guide dan Cluster Mode Overview. https://spark.apache.org/docs/latest/

[3] Dean, J. dan Ghemawat, S. (2004). MapReduce: Simplified Data Processing on Large Clusters. OSDI '04.

[4] Damji, J., et al. (2020). Learning Spark, 2nd Edition. O'Reilly Media.